Lock-Free, SPSC Order Book Matching Engine

Final Project Report

Field	Details
Course	Operating Systems
Submission Type	Project Proposal + Final Report
Team Size	5 members
Language / Platform	C++20 / Linux
Repository	`matching-engine`

Team Members

Member	Name	Roll No.
A	Piyush Satiya	202451130
B	Prayag Kirar	202451130
C	Sanskar Sharma	202451148
D	Sorabh Baldeva	202451156
E	Ved Gupta	202451166

1. Project Overview

We implemented a **Concurrent, Low-Latency Order Matching Engine** — the core of any stock exchange or electronic trading platform. It accepts a stream of buy and sell orders, matches compatible orders under price-time priority, and produces trades.

Although this looks like a finance problem, it is fundamentally an **operating systems problem**: many producers generate events simultaneously, compete for CPU time and memory, and face strict latency requirements. Synchronization, scheduling, memory management, and I/O each appear for a real, measurable reason.

The Central Question

> *What happens to latency and tail latency when we systematically change the underlying OS mechanisms — synchronization, memory allocation, CPU scheduling, and I/O — and can we explain the results?*

This question, not throughput maximization, is the purpose of the project. The same structure (many producers → queue → consumer → results) appears in telemetry pipelines, packet processors, game servers, and messaging infrastructure; a matching engine gives us a concrete, deterministic workload with strict ordering to study it.

---

2. Background

Order book. Stores all outstanding orders by price. A *buy* order means “buy up to *quantity* at *price* or better”; a *sell* order is the mirror. When a new order **crosses** a resting order on the opposite side, a trade occurs at the resting order’s price.

```
        SELL SIDE                Resting:  SELL 5 @ 101
Price       Qty                  Incoming: BUY  5 @ 102
103          20                  Result:   TRADE 5 @ 101
102          15
101           8
-----------------
 99          10
 98          20
       BUY SIDE
```

Price-time priority. (1) Better price first; (2) among equal prices, earlier order first (FIFO per level).

Matching engine. Owns the order book and processes orders one at a time. It is deliberately **single-threaded**: the engineering problem lies in what surrounds it — contention-free delivery, predictable memory management, and scheduler configuration.

---

3. System Architecture

Each layer is independently replaceable, which enables controlled A/B experiments.

```
ORDER PRODUCERS (client threads / network)
                  |
NETWORK INGESTION:  epoll  OR  io_uring
                  |
RING BUFFER:  Mutex-based  OR  Lock-free SPSC
                  |
MATCHING ENGINE (single-threaded)
  ORDER BOOK: std::map + std::deque (price -> FIFO queue)
                  |
TRADE OUTPUT
```

Key decision: a multi-threaded order book would require fine-grained locking on the most latency-sensitive path and is a substantially harder research problem. Concurrency therefore exists only at the **ingestion boundary**. More threads do not automatically mean more performance.

---

4. Component Design and Implementation

4.1 Matching Engine Core

OS concept: deterministic sequential execution, data structures

Each **order** carries an ID, side, fixed-point price, quantity, and a creation timestamp (`CLOCK_MONOTONIC` , ns). Each **trade** records the buy and sell order IDs, price, and quantity.

The book keeps two ordered maps from price to a FIFO queue of orders: the buy side sorted descending and the sell side ascending. This way the first entry of each map is always the best bid or best ask, so no scanning is needed during matching.

Matching (buy order; sell is symmetric): while the incoming order has quantity left and the sell side is non-empty, compare against the best ask. If the incoming price is below it, stop (no cross). Otherwise fill the smaller of the two quantities at the resting price, emit a trade, and reduce both quantities. Remove filled resting orders, and erase price levels that become empty. Any remaining quantity rests at the back of its price level on the buy side.

Correctness tests (required before integration): (1) buy on empty book rests; (2) exact match consumes both orders; (3) partial fill leaves remainder resting; (4) match across multiple levels fills best price first; (5) FIFO at same price; (6) no cross → rests, no trade.

4.2 Ring Buffer: Mutex-Based

OS concept: mutual exclusion, condition variables

The concurrent baseline: a fixed-size circular array guarded by `std::mutex`, with `not_full` / `not_empty` condition variables. Producers block when full; the engine blocks when empty.

Metrics: throughput, p50/p99/p999, CPU utilization, queue-full/empty events, context switches (`perf stat`).

4.3 Ring Buffer: Lock-Free SPSC

OS concept: atomics, memory ordering, lock-free concurrency

Only the producer writes `tail` and only the consumer writes `head`, so no two threads write the same atomic and no compare-and-swap loop is needed. `head` and `tail` sit on separate cache lines (`alignas(64)`) to avoid **false sharing**, where a write to one index invalidates the cache line holding the other.

Memory ordering: the producer writes the slot, then publishes it with a `release` store on `tail`. The consumer's `acquire` load of `tail` pairs with it, guaranteeing the slot's contents are visible before the new index is. The same pairing applies to `head` in the other direction. This is the minimum ordering required and is cheaper than `seq_cst`.

Correctness gate before any benchmarking: sustained stress test (N ≫ buffer size) with no loss or duplication, ordering preserved, and ThreadSanitizer clean.

4.4 Slab Allocator

OS concept: memory management, fragmentation, allocation policy

glibc `malloc` / `new` is general-purpose: it handles arbitrary sizes, manages arenas under concurrency, and consolidates free blocks, producing **non-deterministic latency** that surfaces in p99/p999. The slab allocator pre-allocates one block, carves it into `sizeof(Order)` chunks, and tracks free slots with an **in-place free list** (the first bytes of each free slot point to the next), requiring no bookkeeping memory. Allocation pops the head of the list; deallocation pushes the slot back.

Metrics: allocation/deallocation latency, p999 matching latency, variance. Our hypothesis is not merely that slab is faster, but that it **reduces variability**.

4.5 CPU Scheduling and Affinity

OS concept: scheduling, thread migration, real-time policies

- **Affinity** (`pthread_setaffinity_np`): by default Linux may migrate a thread at any time, causing TLB flushes, cache warm-up, and jitter. Pinning the engine thread to a dedicated core removes migrations (verified via `perf stat -e migrations`).
- **Real-time policy** (`SCHED_FIFO` , priority 50): the thread cannot be preempted by normal `SCHED_OTHER` threads. Requires `CAP_SYS_NICE` or root — verified in Week 1.

Config	Scheduler	Affinity
Baseline	`SCHED_OTHER`	None
Pinned	`SCHED_OTHER`	Core 2
RT	`SCHED_FIFO`	Core 2

Focus is on tail latency (p999, max), since scheduler jitter is a tail phenomenon.

4.6 Network Ingestion

OS concept: I/O subsystems, syscall overhead, kernel/user boundary

Both implementations expose the same interface to the ring buffer.

	`epoll` (readiness-based)	`io_uring` (completion-based)
Flow	`epoll_wait()` → `read()` → parse → push	submit read SQEs → kernel works → reap CQEs → parse → push
Syscalls	≥ 2 per order	Batched via shared submission/completion rings; near zero under load

Controlled variables: client count, order rate and distribution, message size, CPU config, ring buffer type, matching logic. The question is not "which is faster" but *under what workload does io_uring help, and by what mechanism* (syscall counts via `perf stat`).

4.7 Benchmarking Harness

OS concept: performance measurement

A single shared tool so every comparison uses the same measurement mechanism. Each order is timestamped at creation (T1, `CLOCK_MONOTONIC`); latency is T2 − T1 at trade/rest.

Reported per run: **p50, p99, p999, max, throughput**, written to a common CSV schema. Percentiles are used because an average can look excellent while hiding catastrophic tail behavior; scheduler jitter, allocator variance, and lock contention appear in the tail. A small p99→p999 gap indicates a consistent system; a large gap indicates occasional spikes (scheduler interference, allocation stalls, cache misses).

---

5. Methodology

> **Build a correct system → measure → change one subsystem → measure again → explain the difference → keep or reject.**

Core rule: never change multiple variables at once unless studying their interaction. Comparing *Mutex + epoll + SCHED_OTHER* against *Lock-free + io_uring + SCHED_FIFO* cannot attribute any gain to a single change.

Measurement protocol (per configuration):

1. Build the exact configuration under test.
2. Warm up; **discard the first 10,000 orders**.
3. Fixed-rate workload with the same random seed across configs.

4. Timestamp at creation; record latency to trade/rest.
5. **Repeat 10 times**; report the median; flag runs where p999 deviates >20% from the median.
6. Log CPU model, core count, kernel version, governor, SMT status, `liburing` version.
7. Investigate anomalies before reporting; store **raw per-order samples**.

Environment controls: `performance` CPU governor, turbo disabled, core 2 isolated (`isolcpus=2`).

---

6. Experiments

Each experiment changes exactly one variable.

#	Experiment	Variable	Hypothesis / Key Metric
1	Single-threaded baseline	None (direct calls, no threads/network)	Sets the latency floor: p50/p99/p999 of matching alone
2	Mutex vs. lock-free queue	Synchronization	Similar at low contention; mutex adds context switches at high contention. Metric: p999
3	Producer scaling (1, 2, 4, 8)	Producer count	Where does each queue saturate?
4	`new` / `delete` vs. slab	Allocator	p50 similar; p999 lower and more consistent with slab
5	Floating vs. pinned thread	Affinity	Fewer migrations (`perf stat`) correlate with lower p999
6	`SCHED_OTHER` / pinned / pinned + `SCHED_FIFO`	Scheduling	FIFO lowers max and p999, at the cost of starvation risk. A null result is valid
7	epoll vs. io_uring	I/O mechanism	A syscall-reduction story, not only wall-clock

#	Experiment	Variable	Hypothesis / Key Metric
8	Full 8-configuration matrix	Queue × Scheduling × Network	Lowest, most consistent p999, explained from first principles

The 8-configuration matrix (Queue: Mutex / Lock-free; Scheduling: Default / Pinned + `SCHED_FIFO`; Network: epoll / io_uring) is run through the shared harness so results are directly comparable.

Profiling. For every significant difference, `perf` is used to find the *mechanism* (context switches, lock contention, cache behavior, syscall counts), not just confirm the number. An unexplained chart is worth less than two correct sentences of reasoning backed by a measurement.

Final configuration is chosen from measured results, not by which sounds most advanced.

---

7. OS Concepts Covered

Concept	Where It Appears
Processes and threads	Producer, matching engine, and network threads
Mutual exclusion, condition variables	Mutex ring buffer
Atomics, memory ordering, false sharing	SPSC queue (`acquire` / `release` , `alignas(64)`)
Race conditions	Detected by ThreadSanitizer; prevented by atomic ordering
CPU scheduling and affinity	`SCHED_OTHER` vs. `SCHED_FIFO` ; `pthread_setaffinity_np`
Context switching	Measured with `perf stat` , correlated with latency
Memory management	Slab allocator vs. `malloc`
I/O subsystems and system calls	`epoll` vs. `io_uring` ; syscall counts

Concept	Where It Appears
Linux profiling	`perf stat`, `perf record`, `perf report`

---

8. Tools and Technologies

C++20 · Linux (kernel 5.10+) · CMake 3.20+ · GCC 11+/Clang 14+ · `std::thread`, `pthread` · `std::atomic` · `epoll`, `liburing` · ASan, UBSan, TSan · GoogleTest · `perf` · `clock_gettime(CLOCK_MONOTONIC)` · Docker · Git (feature branches)

---

9. Timeline

Week	Focus	Gate
1	Infrastructure: CMake, Docker, sanitizers, tests, harness format	Everyone can build and run a stub
2	Correct single-threaded matching engine	**Hard gate: merge to `main`**
3–5	Parallel development: ring buffer, allocator, scheduling, network	Baseline + optimized version of each
6	Integration	All 8 configs run
7	Integrated benchmarking and profiling	8-config results collected
8	Analysis, final configuration, report	Submission

After the Week 2 gate, components are developed in parallel against the frozen core; only integration and profiling are serial.

---

10. Deliverables

Software: matching engine; mutex and lock-free SPSC ring buffers (compile-time switch); slab allocator; scheduling configuration; epoll and io_uring network layers; benchmarking

harness; unit tests; build system (`CMakeLists.txt`); container config (`Dockerfile` , `docker-compose.yml`).

Report: architecture, OS concept mapping, methodology, p50/p99/p999 for every configuration, latency CDF plots, `perf` output with analysis, mechanism-level explanations, limitations, and future work.

---

11. Scope

In scope: single-threaded order book with SPSC queues; mutex vs. lock-free; slab vs. general-purpose allocator; affinity and real-time scheduling; epoll vs. io_uring; latency distributions; `perf` profiling.

Out of scope: multi-threaded order book, MPMC queues, NUMA-aware design, kernel bypass (DPDK/RDMA), FPGA, SIMD matching, distributed matching, real exchange protocols (FIX), custom network stack — each is either a substantially harder research problem or outside the OS learning objective.

---

12. Success Criteria and Learning Outcomes

The project succeeds if we demonstrate: correct matching under extensive tests; ThreadSanitizer-clean concurrent ingestion; working, measured mutex and lock-free queues; reproducible results over 10 runs per configuration; latency distributions rather than averages; and at least one performance effect with a mechanism-level explanation, together giving a defensible answer to the central question.

> **Wrong:** “Our implementation is X% faster.” **Right:** “Under workload Y, design A behaves differently from design B because of mechanism Z, and our measurements — including p999 distributions and `perf stat` output — support that explanation.”

Learning outcomes. Each member can explain: how price-time priority works; why shared state needs synchronization; why a single-threaded state owner can beat a heavily locked design; how ring buffers and atomics work, including `acquire` / `release` ordering; how ThreadSanitizer detects races; how contention scales with producers; why tail latency matters more than averages; and why every optimization must be justified by measurement.

---

13. Future Extensions

MPSC lock-free queue · cache-line padding experiments · write-ahead log · deterministic workload replay · NUMA experiments · batch processing · custom TCP client · multiple matching partitions (one engine per instrument).